

INTERNAL PROJECT HANDOVER

AI Visual Search — Zen Cart Locomotive Inventory (*conductor.fun*)

Client	USA	Owner	Ahmed Ali, Datix AI
Assigned to	--	Est. duration	4 days

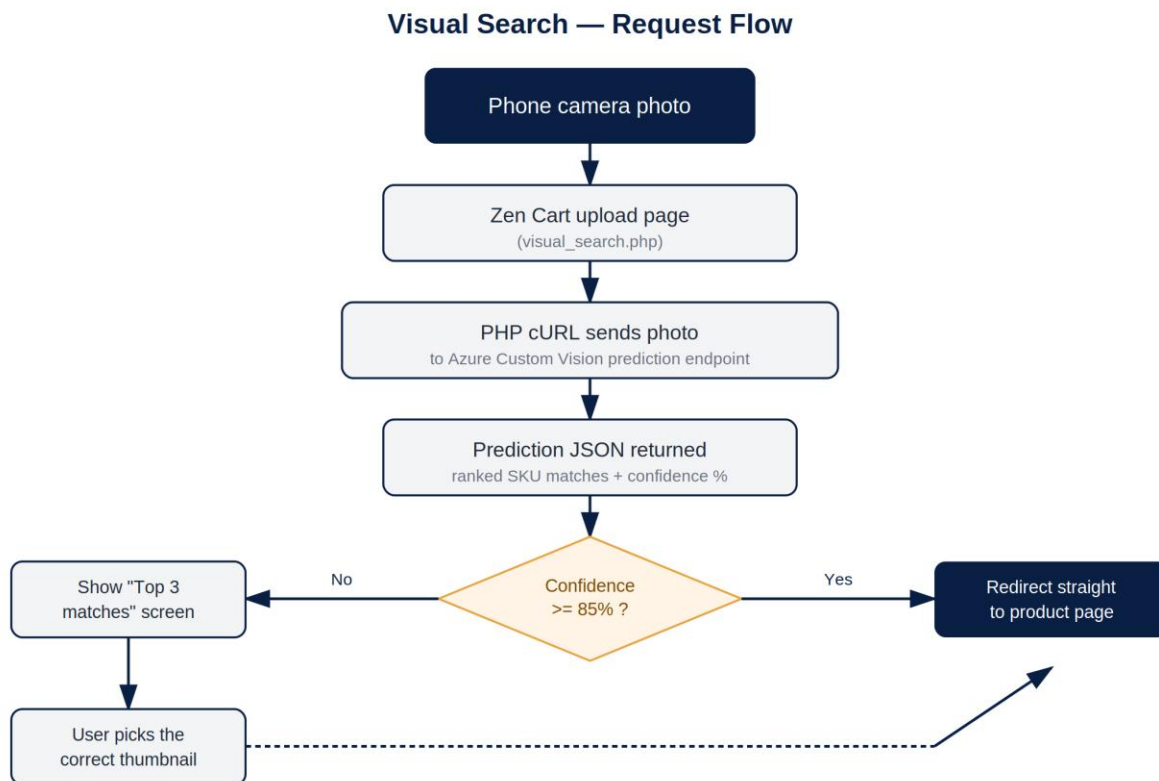
1. What This Project Is

The client runs a private Zen Cart (PHP/MySQL) site as an inventory catalog for a train collection not a live store, checkout is disabled. Visitors browse, add items to a cart, and email the list to the client for preparation. Locomotives are currently looked up via QR code stickers linking to their product page.

Goal: add a Visual Search feature — someone photographs a locomotive with a phone, uploads it on the site, and the system identifies it and opens the matching product page. This is an addition to the site, not a replacement for the QR system, which stays as the reliable fallback.

2. System Architecture — Request Flow

This is the flow for every single visual search request, once the feature is live:



Both paths end at the correct locomotive's product page

The confidence threshold (85% used here as a starting point) is tunable — Step 6 (Real-World Testing) is where the right number for this dataset gets confirmed.

3. Access & Credentials

Handle these credentials securely.

Do not paste them into chat tools, tickets, or shared docs beyond this handover. Use a password manager if you're storing them for the duration of the project, and avoid emailing them further. Once the project is live, ask the client whether they'd like the password rotated.

System	Username	Password
WHM (if needed)	harryusa	--
cPanel	harryusa	--
Zen Cart Admin	test	--

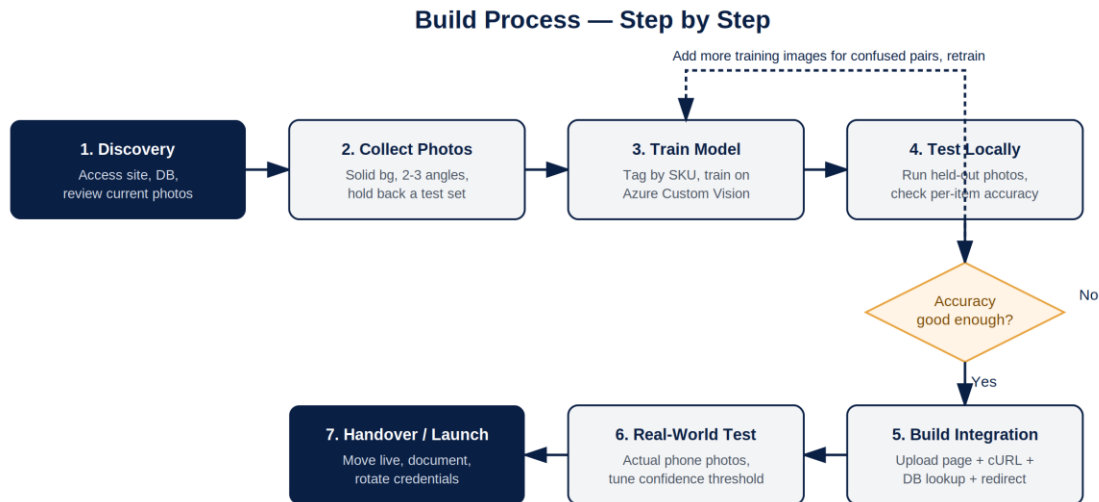
- Site / FTP host: cart.lackawannarailroad.com
- FTP: no dedicated FTP account exists yet — create one from inside cPanel (cPanel → Files → FTP Accounts) rather than using the main cPanel login for file transfers.
- Live Zen Cart admin panel: reachable via the site's /admin path once logged into cPanel — confirm exact admin URL once inside.
- WHM is only needed if server-level access is required (e.g. PHP version changes, server config) — cPanel is enough for most of this work.

4. Recommended Working Setup

1. Log into cPanel first. Create a dedicated FTP account (don't reuse the main login for file transfers) under Files → FTP Accounts.
2. Decide staging vs. live: if possible, create a staging subdomain (e.g. staging.cart.lackawannarailroad.com) and clone the site there via cPanel's site tools, or a manual copy. Build and test the new visual search page on staging first, then move it to live once working.
3. Pull a local copy of the codebase via the new FTP account (FileZilla or similar) so you can work in your own editor rather than editing live files directly in cPanel's File Manager.
4. Open phpMyAdmin (inside cPanel) and locate the products table — note the exact column names for product ID, model/SKU, and existing image paths. You'll need these to map AI predictions back to a product page.
5. Log into the Zen Cart admin panel (test / Conductor62) to see how products/categories are structured from the admin side, and to sanity-check the current photo set already on the live catalog.

5. Step-by-Step Build Plan

Visual overview of the full build sequence, including the retrain loop at Step 4:



Step 1 — Discovery

- Confirm PHP version and Zen Cart version (visible in cPanel / admin footer).
- Review current product photos — most are on plain wood/floor backgrounds (not ideal for training), so a reshoot is expected.
- Confirm the products table structure and how product images are currently linked.

Step 2 — Collect training photos

- Ask the client for reshoot photos per locomotive: solid/white background, even lighting, 2–3 angles each.
- Organize photos into folders named by SKU/product ID before uploading anywhere — this makes tagging much faster.
- Hold back 2–3 photos per locomotive from training — this becomes your local test set later.

Step 3 — Set up the AI model (Azure Custom Vision)

- Create a free Azure Custom Vision project (free tier covers this volume — see cost section of the client proposal).
- Upload training photos, tag each by SKU/product ID, and train the model.
- Review the per-class accuracy report Azure provides after training — this shows which locomotives the model already confuses with each other.

Step 4 — Test locally before touching the live site

- Run the held-out test photos (from Step 2) against the trained model via the API.
- Note which locomotives return low-confidence or wrong matches — these are the ones needing more training images.

- Add extra images for confused pairs and retrain if needed. Repeat until accuracy looks solid on the held-out set.

Step 5 — Build the Zen Cart integration

- Create a new page (e.g. `visual_search.php`) with a simple photo upload form.
- On upload, use PHP `cURL` to send the photo to the Azure prediction endpoint.
- Parse the returned prediction(s): if confidence is high, redirect straight to that product's page (look up `product_id` from your products table).
- If confidence is low, show a small 'Top matches' results page instead of guessing — a few thumbnail options for the user to pick from.
- Link to this new page from the main navigation or product listing, wherever the client wants it visible.

Step 6 — Real-world testing

- Test using actual phone photos taken casually (not studio-quality) — this is where accuracy typically drops from the clean training/test numbers.
- Adjust the confidence threshold that decides 'direct redirect' vs. 'show top matches' based on what you see.
- Test on both the client's original QR-linked items and any newly added ones to confirm nothing in the existing site broke.

Step 7 — Handover

- Move the finished feature from staging to live (if staging was used).
- Write a short note for the client explaining how to add new locomotives to the visual search index in future (i.e., how to add more training photos and retrain).
- Confirm with the client whether to rotate the shared passwords now that access is no longer needed.

6. Recommended AI Models & Platform Comparison

Several AI image-recognition platforms could power this feature. All work the same way conceptually — train a classifier on tagged locomotive photos, call it via an API. Here's how the realistic options compare:

Platform	Model basis	Best for	Cost @ ~50/mo	Verdict
Azure Custom Vision	MobileNet-based classifier (Compact domain)	Small, tagged image sets; simplest setup and training UI	Free tier — likely \$0/mo	Recommended — start here
Google Vertex AI (AutoML Vision)	AutoML-selected CNN backbone	Larger, steadier-traffic deployments	~\$1–1.50/hr endpoint uptime, even idle	Skip — hourly billing is a poor fit here
AWS Rekognition Custom Labels	CNN classifier, AWS-managed	AWS-native shops already using S3/Lambda	Several \$/hr endpoint uptime	Skip — same issue as Vertex

Self-hosted (MobileNetV2 / EfficientNetB0 transfer learning)	Fine-tuned CNN, trained in Python/Keras	Avoiding all third-party API dependency long-term	\$0 API fees, but needs a small VPS (~\$5–10/mo) to host inference	Optional later upgrade, not needed to start
Claude API (general vision-language model)	General-purpose vision-language model, not a trained classifier	Prototyping, or comparing a query photo against a small reference set	Pay-per-token, likely < \$5/mo at this volume	Fallback/comparison option, not the primary build

Recommendation: build on Azure Custom Vision. It is purpose-trained on the client's own images (rather than general-purpose), has no always-on hourly cost, and its free tier comfortably covers this traffic level indefinitely.

If Azure's accuracy plateaus on the hardest near-identical pairs even after retraining, the self-hosted transfer-learning option (MobileNetV2/EfficientNetB0) is worth revisiting — a custom-trained model with full control over the training process can sometimes edge out a general-purpose managed service on very fine-grained distinctions, at the cost of needing to host and maintain the inference server yourself.

7. Technical Implementation Details

Zen Cart integration point

- New file: /visual_search.php at the site root, following the same header/footer include pattern as existing Zen Cart pages so it matches the site's theme.
- Form: a single file input (accept="image/*", with the capture attribute so mobile browsers open the camera directly) plus a submit button.
- On submit, the photo posts to the same page (or a small handler script), which performs the API call server-side — never call the Azure API directly from the browser, to keep the API key off the client.

Minimal request flow (PHP + cURL)

```
$ch =
curl_init('https://<endpoint>.cognitiveservices.azure.com/customvision/v3.0/Prediction/<projectId>
/classify/iterations/<iterationName>/image');
curl_setopt($ch, CURLOPT_POST, true);
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    'Prediction-Key: <your-key>',
    'Content-Type: application/octet-stream'
]);
curl_setopt($ch, CURLOPT_POSTFIELDS, file_get_contents($_FILES['photo']['tmp_name']));
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
$response = json_decode(curl_exec($ch), true);
curl_close($ch);

$topMatch = $response['predictions'][0];
if ($topMatch['probability'] >= 0.85) {
```

```
// look up products_id where products_model = $topMatch['tagName']
// header('Location: index.php?main_page=product_info&products_id=' . $productId);
} else {
    // show top 3 predictions as a picker instead of guessing
}
```

Database mapping

- The tag name used when training each image in Azure should exactly match a unique, stable value already in the products table — the SKU/products_model field is the natural choice, since it won't change even if a product's name or description is edited later.
- Keep a simple lookup: `SELECT products_id FROM products WHERE products_model = '<tagName>' LIMIT 1` — this is the only DB query the integration needs.

Confidence thresholds (starting point, tune during Step 6)

- $\geq 85\%$: redirect straight to the product page.
- 50–85%: show a "Top 3 matches" picker with thumbnails, letting the user confirm.
- $< 50\%$: show a friendly "No confident match — browse the catalog" message rather than a picker full of unlikely guesses.

Image handling & basic safeguards

- Resize/compress the uploaded photo server-side before sending to the API (e.g. max 1024px on the longest edge) — keeps requests fast and avoids unnecessary API costs on oversized phone photos.
- Correct EXIF orientation before processing — phone photos are frequently saved sideways/upside-down relative to how they display.
- Validate file type and size server-side (accept jpg/png only, reasonable max size, e.g. 10MB) before forwarding anything to the API.
- No need for user accounts or rate limiting given the private, low-traffic nature of this site — but keep the Azure prediction key server-side only, never exposed in page source.

8. Tools You'll Need

- FTP client (FileZilla or similar) for pulling/pushing code.
- A code editor (VS Code recommended) for working on the PHP locally.
- Azure account for Custom Vision (free tier).
- Access to phpMyAdmin via cPanel for database inspection.
- A phone to take/test real-world photos against the live feature.

9. Known Constraints

- This is shared cPanel hosting — no GPU, no local ML runtime. The AI model must stay cloud-hosted (Azure); do not attempt to run inference directly on the server.

- Visual search will not be perfectly accurate for near-identical locomotives — this is expected and handled via the 'top matches' fallback, not something to over-engineer around.
- The existing QR code system must not be removed or broken by this work — it remains the reliable fallback lookup method.

10. Questions to Confirm With the Client Before Starting

1. Staging subdomain first, or work directly on the live site?
2. Roughly how many total locomotives need training photos?
3. Any locomotives already known to look nearly identical to each other?
4. Should the Azure account be created under the client's own billing, or ours for now?
5. Any target date this needs to be live by?

Questions on any step — reach out before making changes to the live site.

Ahmed Ali

Founder & CEO
Datix AI